

BOOGLE

Slang Dictionary Application Documentation & Source Code Explanation

Data Structures — AOL Case Study

Individual Project | Binus University

Data Structure Used: Trie (Prefix Tree)

Language: C++ (.cpp) using C standard libraries only

Libraries: <stdio.h>, <stdlib.h>, <string.h>, <ctype.h>

No regex used. All validation done with manual character-by-character logic.

1. Application Overview

Boogle is a command-line application that acts as a dictionary for internet slang words. It allows users to add new slang words with their definitions, search for existing words, browse by prefix, and list all stored words. All words are stored in a **Trie** (also called a Prefix Tree), which is a specialized tree data structure ideal for string-based operations like prefix search and lexicographic ordering.

1.1 Why Trie?

A Trie stores strings character by character. Each node represents one character, and a path from root to a marked node spells out a complete word. This makes:

- **Insert/Search $O(L)$** — L = word length, very fast regardless of dictionary size.
- **Prefix search natural** — just navigate to the prefix node, then DFS the subtree.
- **Lexicographic order free** — iterating children index 0–25 (a–z) automatically gives sorted output.

1.2 Application Menu

Menu 1	Release a new slang word — insert or update a word in the Trie.
Menu 2	Search a slang word — find an exact word and show its definition.
Menu 3	View all slang words starting with a prefix — DFS from prefix node.
Menu 4	View all slang words — full DFS traversal, lexicographic output.
Menu 5	Exit — terminate the program gracefully.

2. Data Structure: TrieNode

The entire dictionary is built from **TrieNode** structs linked together. Below is the struct definition and its explanation:

struct TrieNode (boogle.cpp)

```
#define ALPHABET_SIZE 26      // supports lowercase a-z only
#define MAX_WORD_LEN 100     // max characters per word
#define MAX_DESC_LEN 500     // max characters per description
#define MAX_RESULTS 1000     // max words collectible in one traversal

typedef struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE]; // child pointers: index 0='a', 25='z'
    int isEndOfWord; // flag: 1 = a complete word ends at this node
    char* description; // heap-allocated string for the word's meaning
} TrieNode;
```

Field Explanations

children[26]	An array of 26 pointers, one for each letter a–z. children[0] = branch for 'a', children[1] = branch for 'b', ..., children[25] = branch for 'z'. A NULL pointer means no word passes through that letter at this node.
isEndOfWord	An integer flag. When set to 1, it means the path from the root to this node spells a complete, valid slang word in the dictionary. A node can be both an end-of-word AND have children (e.g., both 'da' and 'dadan' can exist simultaneously).
description	A heap-allocated (malloc) string storing the definition of the slang word. Only meaningful when isEndOfWord == 1. Using a pointer (not a fixed array) saves memory since most nodes in the trie are intermediate letters, not word endpoints.

createNode() — Allocating a new empty node

```
TrieNode* createNode() {
    // malloc: allocate memory on the heap for one TrieNode
    TrieNode* node = (TrieNode*) malloc(sizeof(TrieNode));

    node->isEndOfWord = 0; // not a word endpoint yet
    node->description = NULL; // no description yet

    // initialize all 26 child pointers to NULL (no branches yet)
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;

    return node;
}
```

Every new node starts with all children NULL and isEndOfWord=0, representing a 'blank' intermediate character in the trie.

3. Input & Validation Utilities

Since the assignment prohibits regex, all input validation is done manually using simple loops and character comparisons. Below are the utility functions:

3.1 readLine()

readLine() — Safe single-line input

```
void readLine(char* buf, int maxLen) {
    // fgets reads at most maxLen-1 chars + null terminator,
    // stopping at newline or EOF – safe, no buffer overflow
    if (fgets(buf, maxLen, stdin) != NULL) {
        int len = strlen(buf);
        // fgets keeps the '\n' character – we strip it
        if (len > 0 && buf[len - 1] == '\n')
            buf[len - 1] = '\0';
    }
}
```

Purpose: Read one line of user input safely. Using fgets instead of scanf prevents buffer overflow and correctly handles spaces within input.

3.2 toLowerStr()

toLowerStr() — Case normalization

```
void toLowerStr(char* s) {
    // iterate every character in the string
    for (int i = 0; s[i]; i++)
        // tolower() converts uppercase letters to lowercase;
        // cast to unsigned char avoids undefined behavior on non-ASCII
        s[i] = tolower((unsigned char) s[i]);
}
```

Purpose: Normalize all slang word inputs to lowercase so that 'Rizz', 'RIZZ', and 'rizz' are treated as the same word. Applied before every Trie operation.

3.3 hasSpace()

hasSpace() — Detect spaces without regex

```
int hasSpace(const char* s) {
    // scan every character manually
    for (int i = 0; s[i]; i++)
        if (s[i] == ' ') return 1; // found a space → invalid
    return 0;                       // no space found → valid
}
```

Purpose: Slang words must not contain spaces. This replaces a regex like '\\s' with a simple character scan — O(n) time, zero dependencies.

3.4 countWords()

countWords() — Word counter using state machine

```
int countWords(const char* s) {
    int count = 0, inWord = 0;
    for (int i = 0; s[i]; i++) {
        if (s[i] != ' ') {
            if (!inWord) { // transition: space→word boundary
                count++;
                inWord = 1;
            }
        } else {
            inWord = 0; // transition: word→space boundary
        }
    }
    return count;
}
```

Purpose: Count how many words are in the description string. The description must be more than 1 word (i.e., `countWords > 1`). Uses a two-state machine (inside-word / outside-word) — correctly handles multiple consecutive spaces.

3.5 pressEnter()

pressEnter() — Pause before returning to menu

```
void pressEnter() {
    printf("Press enter to continue...");
    fflush(stdout); // ensure the message appears before blocking
    int c;
    // consume characters until newline (Enter) or end-of-file
    while ((c = getchar()) != '\n' && c != EOF);
}
```

Purpose: After displaying a result, pause execution and wait for the user to press Enter before redrawing the main menu. `fflush(stdout)` guarantees the prompt is visible before the program blocks on `getchar()`.

4. Trie Core Operations

4.1 trieInsert() — Insert or Update

trieInsert()

```
int trieInsert(TrieNode* root, const char* word, const char* desc) {
    TrieNode* curr = root;

    for (int i = 0; word[i]; i++) {
        int idx = word[i] - 'a';    // map char to index: 'a'=0, 'b'=1, ..., 'z'=25

        // if no branch for this letter, create one
        if (curr->children[idx] == NULL)
            curr->children[idx] = createNode();

        curr = curr->children[idx]; // move deeper into the trie
    }

    int isNew = !curr->isEndOfWord; // remember if this is a brand-new word
    curr->isEndOfWord = 1;           // mark this node as a complete word

    // free old description (if updating) and store the new one
    if (curr->description) free(curr->description);
    curr->description = (char*) malloc(strlen(desc) + 1);
    strcpy(curr->description, desc);

    return isNew; // 1 = new word inserted, 0 = existing word updated
}
```

Logic: For each character in the word, compute its index (0–25) and follow or create the corresponding child pointer. After consuming all characters, the current node is the word's endpoint — mark it and store the description. If the node was already an endpoint, this is an update (returns 0); otherwise it's a new insertion (returns 1).

4.2 trieSearch() — Exact Word Lookup

trieSearch()

```
TrieNode* trieSearch(TrieNode* root, const char* word) {
    TrieNode* curr = root;

    for (int i = 0; word[i]; i++) {
        int idx = word[i] - 'a';

        // if any character's branch is missing → word doesn't exist
        if (curr->children[idx] == NULL) return NULL;

        curr = curr->children[idx];
    }

    // we reached the end of the word — only valid if this IS a word endpoint
    return (curr->isEndOfWord) ? curr : NULL;
}
```

Logic: Navigate the trie following the word's characters. If any branch is missing, the word cannot exist → return NULL. If we reach the end, the word exists only if `isEndOfWord == 1` (otherwise it's just a prefix of another word, e.g. searching 'da' when only 'dadan' is stored would reach the 'a' node but `isEndOfWord` would be 0).

4.3 trieGetPrefixNode() — Navigate to Prefix

trieGetPrefixNode()

```
TrieNode* trieGetPrefixNode(TrieNode* root, const char* prefix) {
    TrieNode* curr = root;

    for (int i = 0; prefix[i]; i++) {
        int idx = prefix[i] - 'a';
        if (curr->children[idx] == NULL) return NULL; // prefix not in trie
        curr = curr->children[idx];
    }

    return curr; // returns the node at the END of the prefix
}
```

Logic: Same traversal as `trieSearch`, but does NOT check `isEndOfWord` at the end. Returns the node where the prefix ends — from this node, `collectWords()` can DFS to find all words sharing that prefix.

4.4 collectWords() — DFS Lexicographic Collection

collectWords()

```
void collectWords(TrieNode* node, char* prefix, int depth,
                  char results[][MAX_WORD_LEN], int* cnt) {
    if (*cnt >= MAX_RESULTS) return; // safety cap

    if (node->isEndOfWord) {          // this node is a complete word
        prefix[depth] = '\0';         // null-terminate the accumulated string
        strcpy(results[*cnt], prefix); // save word to results array
        (*cnt)++;                     // increment counter via pointer
    }

    // iterate children a-z (index 0-25) – this guarantees lexicographic order
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (node->children[i]) {
            prefix[depth] = 'a' + i; // append the next character
            collectWords(node->children[i], prefix, depth + 1, results, cnt);
        }
    }
}
```

Logic: Depth-First Search (DFS) starting from any given node. The prefix buffer accumulates characters as we go deeper, and is unwound automatically as recursion returns. Iterating children in index order 0→25 means 'a' is always visited before 'b', etc. — producing lexicographically sorted output without any sorting step.

Note: The 'depth' parameter tracks how deep we are, which directly corresponds to the buffer index where we should write the next character.

5. Menu Function Explanations

5.1 menuReleaseWord() — Menu 1

Handles inserting a new slang word into the Trie.

Step 1	Loop prompting until a valid slang word is entered: length > 1 AND no spaces.
Step 2	Apply toLowerStr() to normalize the input to lowercase.
Step 3	Loop prompting until a valid description is entered: more than 1 word (countWords > 1).
Step 4	Call trieInsert(). If it returns 1, print 'released'; if 0, print 'updated'.
Step 5	Call pressEnter() to pause before returning to menu.

5.2 menuSearchWord() — Menu 2

Handles searching for an exact slang word.

Step 1	Loop prompting until a valid word is entered (same validation as Menu 1).
Step 2	Call trieSearch(). Returns the TrieNode if found, NULL if not.
Step 3	If NULL: print 'There is no word ...' message.
Step 4	If found: print the word and its description from node->description.
Step 5	Call pressEnter().

5.3 menuPrefixSearch() — Menu 3

Handles finding all words that start with a given prefix.

Step 1	Read prefix input (no length/space validation — prefix can be any string).
Step 2	Apply toLowerStr() to normalize.
Step 3	Call trieGetPrefixNode() to navigate to the prefix end-node.
Step 4	If NULL: print 'There is no prefix ...' message.
Step 5	If found: call collectWords() starting from that node, with prefix string pre-loaded into buf.
Step 6	Print numbered list of results (already in lexicographic order).

5.4 menuViewAll() — Menu 4

Displays all words in the dictionary.

Step 1	Call collectWords() starting from root, with an empty buffer.
Step 2	If count == 0: print empty dictionary message.
Step 3	Otherwise: print numbered list of all words in lexicographic order.

5.5 main() — Entry Point

main()

```
int main() {
    TrieNode* root = createNode(); // the root node – represents empty string,
                                   // all 5 menus share this single trie root

    int choice;
    char inputBuf[16];              // buffer for menu selection input

    while (1) {                    // infinite loop until user selects Exit
        // display menu...
        readLine(inputBuf, sizeof(inputBuf));
        choice = atoi(inputBuf);    // atoi: safe int parse, returns 0 on invalid

        switch (choice) {
            case 1: menuReleaseWord(root); break;
            case 2: menuSearchWord(root); break;
            case 3: menuPrefixSearch(root); break;
            case 4: menuViewAll(root); break;
            case 5: printf("Thank you... Have a nice day :)\n"); return 0;
            default: printf("Invalid choice. Please try again.\n");
        }
    }
}
```

The root TrieNode is created once and passed to every menu function by pointer. All insertions and queries operate on the same shared trie throughout the session.

6. Known Limitations

- **Lowercase only:** The trie supports only a–z (ALPHABET_SIZE=26). Words with digits, hyphens, or uppercase letters after normalization will cause undefined index behavior. Slang words are assumed to be purely alphabetic.
- **In-memory only:** All data is lost when the program exits. There is no file persistence (not required by the spec).
- **No memory deallocation on exit:** The recursive free of all TrieNodes is omitted for brevity. The OS reclaims all heap memory on process exit, so this is safe in practice.
- **MAX_RESULTS cap:** collectWords() stops at 1000 results. For a realistic slang dictionary this is more than sufficient.
- **Description overwrites on update:** Re-inserting an existing word completely replaces its description. There is no history or version tracking.

7. Custom Test Cases & Results

The following test cases were designed to verify all features and edge cases of the Boogle application. Each test case was run on the compiled program and the terminal output is shown below.

Test Case 1 — Empty Dictionary (Menu 4 before any insert)

Purpose: Verify that Menu 4 shows the empty message when no words have been added yet.

Input sequence: Choose menu 4 → press Enter → choose menu 5 (Exit)

```
===== BOOGLE SLANG DICTIONARY =====
1. Release a new slang word
2. Search a slang word
3. View all slang words starting with a certain prefix word
4. View all slang words
5. Exit
Choose menu: 4
There is no slang word yet in the dictionary.
Press enter to continue...

===== BOOGLE SLANG DICTIONARY =====
Choose menu: 5
Thank you... Have a nice day :)
```

Result: ■ PASS — Correct empty message displayed.

Test Case 2 — Input Validation on Menu 1

Purpose: Verify that the word validation (length > 1, no spaces) and description validation (more than 1 word) both work correctly by rejecting bad inputs first.

Input sequence: Choose 1 → type 'x' (1 char, rejected) → type 'rizz' (valid) → type 'The' (1 word, rejected) → type 'rizz charm charisma aura' (valid)

```
Choose menu: 1
Input a new slang word [Must be more than 1 characters and contains no space]: x
Input a new slang word [Must be more than 1 characters and contains no space]: rizz
Input a new slang word description [Must be more than 2 words]: The
Input a new slang word description [Must be more than 2 words]: rizz charm charisma aura
Successfully released new slang word.
Press enter to continue...
```

Result: ■ PASS — Single-char word rejected. Single-word description rejected. Valid inputs accepted and word inserted.

Test Case 3 — Full Flow: Insert, View All, Prefix, Search, Update, Exit

Purpose: End-to-end test covering all 5 menus with 5 custom slang words.

Words inserted: rizz, simp, slay, npc, ngl

```
[After inserting: rizz, simp, slay, npc, ngl]
```

```
Choose menu: 4
```

```
List of all slang words in the dictionary:
```

1. ngl
2. npc
3. rizz
4. simp
5. slay

```
Press enter to continue...
```

Result: ■ PASS — All 5 words displayed in correct lexicographic (a→z) order.

```
Choose menu: 3
```

```
Input a prefix to be searched: s
```

```
Words starts with "s":
```

1. simp
2. slay

```
Press enter to continue...
```

Result: ■ PASS — Prefix 's' correctly returns 'simp' and 'slay' in order.

```
Choose menu: 2
```

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: ngl
```

```
Slang word : ngl
```

```
Description : not gonna lie honest
```

```
Press enter to continue...
```

Result: ■ PASS — Found word and description displayed correctly.

```
Choose menu: 2
```

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: bruh
```

```
There is no word "bruh" in the dictionary.
```

```
Press enter to continue...
```

Result: ■ PASS — Word not found message shown correctly.

```
Choose menu: 1
```

```
Input a new slang word [Must be more than 1 characters and contains no space]: rizz
```

```
Input a new slang word description [Must be more than 2 words]: updated rizz means natural charm  
Successfully updated a slang word.
```

```
Press enter to continue...
```

```
Choose menu: 2
```

```
Input a slang word to be searched [Must be more than 1 characters and contains no space]: rizz
```

```
Slang word : rizz
```

```
Description : updated rizz means natural charm
```

```
Press enter to continue...
```

Result: ■ PASS — Re-inserting existing word updates description correctly. Search confirms new description is stored.

```
Choose menu: 3
```

```
Input a prefix to be searched: xyz
```

```
There is no prefix "xyz" in the dictionary.  
Press enter to continue...
```

Result: ■ PASS — Non-existent prefix shows correct empty message.

```
Choose menu: 5  
Thank you... Have a nice day :)
```

Result: ■ PASS — Program exits cleanly.

Summary of Test Results

TC1 — Empty Dictionary	■ PASS
TC2 — Word Input Validation	■ PASS
TC2 — Description Validation	■ PASS
TC3 — Insert Multiple Words	■ PASS
TC3 — View All (Lexicographic)	■ PASS
TC3 — Prefix Search Found	■ PASS
TC3 — Prefix Search Not Found	■ PASS
TC3 — Search Found	■ PASS
TC3 — Search Not Found	■ PASS
TC3 — Update Existing Word	■ PASS
TC3 — Exit	■ PASS

8. Complete Source Code (boogle.cpp)

The complete, annotated source code of the Boogle application is listed below.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define ALPHABET_SIZE 26
#define MAX_WORD_LEN 100
#define MAX_DESC_LEN 500
#define MAX_RESULTS 1000

typedef struct TrieNode {
    struct TrieNode* children[ALPHABET_SIZE];
    int isEndOfWord;
    char* description;
} TrieNode;

TrieNode* createNode() {
    TrieNode* node = (TrieNode*) malloc(sizeof(TrieNode));
    node->isEndOfWord = 0;
    node->description = NULL;
    for (int i = 0; i < ALPHABET_SIZE; i++)
        node->children[i] = NULL;
    return node;
}

void readLine(char* buf, int maxLen) {
    if (fgets(buf, maxLen, stdin) != NULL) {
        int len = strlen(buf);
        if (len > 0 && buf[len - 1] == '\n')
            buf[len - 1] = '\0';
    }
}

void toLowerStr(char* s) {
    for (int i = 0; s[i]; i++)
        s[i] = tolower((unsigned char) s[i]);
}

int hasSpace(const char* s) {
    for (int i = 0; s[i]; i++)
        if (s[i] == ' ') return 1;
    return 0;
}

int countWords(const char* s) {
    int count = 0, inWord = 0;
    for (int i = 0; s[i]; i++) {
        if (s[i] != ' ') {
            if (!inWord) { count++; inWord = 1; }
        } else { inWord = 0; }
    }
    return count;
}

void pressEnter() {
    printf("Press enter to continue...");
    fflush(stdout);
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
```


